

K-Optimal and CS-Near-Optimal Exact Synthesis of Two-Qubit Clifford+CS Operators

Xiaoning Bian* and Yuan Feng†

Department of Computer Science and Technology, Tsinghua University

(Dated: June 3, 2026)

We present a method for decomposing a two-qubit Clifford+CS operator (equivalently, a 4×4 unitary with entries in $\mathbb{D}[i] = \mathbb{Z}[\frac{1}{2}, i]$ [4]) into a Clifford+CS circuit without ancillae. Let k denote the least number of K (scaled H) gates needed to implement a given unitary U . Our method decomposes U , up to a global phase, into a circuit of length at most $10k + 9$, using exactly k K gates and at most $k + 1$ CS gates. The K -count is therefore optimal, and the CS-count is at most 2 times optimal. We also improve the lower bound on the optimal CS-count from [17] by a factor of 2. Let l be the least natural number such that the entries of $(1 + i)^l U$ are all Gaussian integers; we prove that $k \in \{2l, 2l - 1, 2l - 2\}$. In experiments, our method always achieves the optimal CS-count and reduces the K -count by 19% on average relative to the best known prior algorithm. Part of our approach adapts the residue analysis of Russell [34] for the Clifford+ T gate set, with two main differences: we give explicit gate-count bounds (e.g., $10k + 9$ rather than $O(k)$, where the two k 's are different but related), and we work over the simpler ring $\mathbb{Z}[\frac{1}{2}, i]$, a subring of the $\mathbb{Z}[\frac{1}{\sqrt{2}}, i]$ used by Russell.

I. INTRODUCTION

In quantum computation, decomposing unitary operators into gates from a fixed universal set is a fundamental problem [32]. The Clifford+Controlled- S gate set (Clifford+CS) is universal, as is the well-studied Clifford+ T gate set. We study the exact synthesis of two-qubit Clifford+CS operators. Prior work on two-qubit Clifford+CS and Clifford+ T synthesis has mainly aimed at minimizing the CS-count or T-count, since these non-Clifford gates are typically the costliest resource. However, work on T-gate optimization [1, 5, 9, 10, 20, 24] has identified the H gate as a bottleneck. As Soeken et al. [1] put it: “It has been shown that T-depth optimizations can be implemented efficiently for circuits consisting only of T and $CNOT$ gates and that H gates impede the optimization significantly.”

We propose a method that produces circuits with a minimal number of K (scaled H) gates and a near-minimal number of CS gates, thereby mitigating the H -gate bottleneck. Simultaneously minimizing two distinct gate counts is also of mathematical interest. Earlier work on single-qubit Clifford+ T synthesis achieved the simultaneous minimum of the Hadamard and T counts [14, 26, 30]; extending this result to the multi-qubit case has remained open.

A. Background

Consider the following unitary operators:

$$i, K = \omega^7 H = \frac{1}{1+i} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}, S = \begin{bmatrix} 1 & 0 \\ 0 & i \end{bmatrix}, \text{ and}$$

$$CZ = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{bmatrix}.$$

Here, i is the complex unit; $\omega = e^{\frac{i\pi}{4}}$ is a primitive 8th root of unity; H is the Hadamard gate and K is a scaled version of H ; S is sometimes called the *phase gate*; and CZ is the controlled- Z gate. Under multiplication, identities, and tensor products, these operators generate the *Clifford group*. The precise group depends on which scalars are included; we include only ± 1 and $\pm i$, so the above gates generate the Clifford group exactly.

Every operator U obtained in this way is of size $2^n \times 2^n$ for some $n \in \mathbb{N}$; we say that U acts on n qubits. For each n , the group of n -qubit Clifford operators is finite (see, e.g., [35]), so the Clifford gate set alone is not universal for quantum computing. Universality is recovered by adding the controlled phase gate

$$CS = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & i \end{bmatrix}.$$

The resulting operators are called Clifford+CS operators; note that $CZ = CS^2$. We write CCS for the group of two-qubit Clifford+CS operators.

We adopt the following notational conventions. Single-qubit gates lifted to two qubits are subscripted by the active qubit: $K_0 = K \otimes I$, $K_1 = I \otimes K$, and likewise for S_0, S_1 , where I is the 2×2 identity. The scalar i is identified with the 4×4 matrix $i(I \otimes I)$. For controlled gates, the last subscript denotes the target qubit; for example, CX_{01} is the controlled- X with control qubit 0 and target qubit 1, abbreviated CX , while CX_{10} is abbreviated XC . Since the controlled- S and controlled- Z gates are symmetric ($CS_{jk} = CS_{kj}$ and $CZ_{jk} = CZ_{kj}$), we simply write CS and CZ .

* bian@mail.tsinghua.edu.cn

† yuan.feng@tsinghua.edu.cn

We also use the Z - and X -gates, the controlled- K gate $CK = I \oplus K$, and the swap gate Ex . These can be expressed in terms of the primitive gates by $Z = S^2$, $X = KZK \cdot i$, $CX = K_1CZK_1 \cdot i$, $Ex = CXXCX$, and

$$CK = CZ Z_0 S_0 Z_1 S_1 K_1 CSK_1 S_1 \cdot i. \quad (1)$$

Equation (1) shows that $CK \in CCS$. As with CX , we shorten CK_{01} to CK and CK_{10} to KC .

Definition I.1 (Two-qubit Clifford+ CS circuit). Let

$$\mathcal{G} = \{X_j, Z_k, S_l, K_m, CZ, CS, CX, XC, Ex\},$$

where $j, k, l, m \in \{0, 1\}$. We call $\mathcal{G}$ the two-qubit Clifford+ CS gate set, and a two-qubit Clifford+ CS circuit a finite sequence of elements of $\mathcal{G}$. We write $\mathcal{CTR}$ for the set of two-qubit Clifford+ CS circuits.

Each circuit $D \in \mathcal{CTR}$ implements a unique operator $[D] \in CCS$, giving a matrix-semantics map $[\cdot] : \mathcal{CTR} \rightarrow CCS$. We adopt the “matrix-multiplication” convention: the semantics map satisfies $[A \cdot B] = [A] \cdot [B]$, which is the reverse of the usual quantum-circuit order. Here the first $\cdot$ denotes circuit concatenation and the second matrix multiplication; either may be omitted when context is clear.

B. Related work

Operator synthesis can be performed exactly or approximately. We restrict attention to exact synthesis and review the most closely related work below.

1. Single-qubit and single-qutrit

For single-qubit Clifford+ T synthesis, [14, 26, 30] achieve the simultaneous minimum use of both the Hadamard and T gates. For the single-qutrit case, [16, 33] achieve the minimum T -count, and [22] addresses the general single-qutrit case. Forest et al. [12] generalize exact synthesis from Clifford+ T to single-qubit Clifford-cyclotomic gate sets, and Kliuchnikov and Yard [28] develop a general algebraic framework for exact synthesis over number rings, unifying several gate sets under one theory.

2. Two-qubit

Three works on two-qubit synthesis guarantee optimality of a non-Clifford gate. Glaudell et al. [17] give a CS -optimal synthesis of Clifford+ CS operators, and Li et al. [29] give a T -near-optimal synthesis of Clifford+ T operators with worst-case T -count at most 10 times optimal. Neither considers H -count optimization, although the former implies an H -count at most

2 times optimal. More recently, Glaudell et al. [15] gave a high-performance exact synthesis framework for two-qubit Clifford+ T circuits that optimizes the T -count exactly, using a meet-in-the-middle strategy with algebraic canonicalization and lookup tables. All three works rely on the exceptional isomorphism $SU(4) \cong \text{Spin}(6)$. Our work, by contrast, does not use this isomorphism; instead, we employ a residue analysis similar to that of Russell [34], but over the simpler ring $\mathbb{Z}[\frac{1}{2}, i] \subset \mathbb{Z}[\frac{1}{\sqrt{2}}, i]$. Given a unitary U , Russell’s method outputs a circuit of gate count $O(k)$, where k is the least natural number such that the entries of $(1 + \omega)^k U$ lie in $\mathbb{Z}[\omega]$.

3. Multi-qubit and multi-qutrit

Multi-qubit Clifford+ T synthesis was introduced by Giles and Selinger [13], whose algorithm has worst-case gate count $O(3^{2^n} nk)$, later improved by Kliuchnikov [25] to $O(4^n nk)$. Glaudell et al. [4] gave exact multi-qubit synthesis for several subclasses of Clifford+ T operators such as Toffoli+Hadamard and Clifford+ CS . In particular, they showed that U is a two-qubit Clifford+ CS operator if and only if U is a 4×4 unitary with entries in $\mathbb{Z}[\frac{1}{2}, i]$. Amy et al. [3] adapted the method of [25] to Toffoli+Hadamard operators with a similar gate-count improvement. None of these multi-qubit synthesis works consider T -count optimality. Kliuchnikov and Schönnenbeck [27] recently developed an optimal-search algorithm for multi-qubit exact synthesis using A^* search; for two-qubit circuits, its runtime is proportional to the T -count. Exact synthesis of multi-qubit Clifford+Cyclotomic operators is treated in [2, 11], and the multi-qutrit case in [19, 23]. Beyond these, [18] proves that qutrit metaplectic gates are a subset of Clifford+ T , and [31] proposes a heuristic algorithm achieving V -gate optimality.

II. SOME ALGEBRA

Before describing the synthesis algorithm, we develop the algebraic groundwork on which it rests. The relevant rings are $\mathbb{Z}[i]$ and $\mathbb{D}[i]$, together with the quotients $\mathbb{Z}[i]/(\gamma^n)$, which provide a fine-grained way to track how operators act on entries.

A. Rings

We write $\mathbb{N} = \{0, 1, \dots\}$ for the natural numbers and $\mathbb{Z}$, $\mathbb{R}$, $\mathbb{C}$ for the integers, reals, and complex numbers, respectively. The complex conjugate of $z \in \mathbb{C}$ is denoted $z^\dagger$. Let $\mathbb{Z}[i] = \{a + bi \mid a, b \in \mathbb{Z}\}$ be the ring of Gaussian integers. As a Euclidean domain [6], $\mathbb{Z}[i]$ supports division with remainder, greatest common divisors, divisibility, and primes (non-units not expressible as products of two non-units). The element $\gamma = 1 + i$ is a Gaussian

prime [21], and since $\gamma^2 = 2i$, the elements γ^2 and 2 are associates. Using Euclidean division, one can verify that

$$\mathbb{Z}[i]/(\gamma) = \{0, 1\}, \quad \mathbb{Z}[i]/(2) = \mathbb{Z}[i]/(\gamma^2) = \{0, 1, i, 1+i\},$$

$$\text{and } \mathbb{Z}[i]/(\gamma^3) = \{\pm 1, \pm i, 0, 1+i, 1-i, 2\}.$$

(Strictly, $\mathbb{Z}[i]/(\gamma)$ denotes the quotient ring of $\mathbb{Z}[i]$ modulo the ideal (γ) ; the sets above list complete systems of residue class representatives, with the implicit $\text{mod } (\gamma^n)$ omitted for brevity.)

Let $\mathbb{D} = \mathbb{Z}[\frac{1}{2}] = \{\frac{a}{2^k} \mid a \in \mathbb{Z}, k \in \mathbb{N}\}$ be the ring of *dyadic fractions*, i.e., fractions whose denominator is a power of 2. Consider the ring $\mathbb{D}[i] = \{r + si \mid r, s \in \mathbb{D}\} = \mathbb{Z}[\frac{1}{2}, i]$. For every $t \in \mathbb{D}[i]$, there exists some $k \in \mathbb{N}$ with $\gamma^k t \in \mathbb{Z}[i]$, motivating the following definition.

Definition II.1. Let $t \in \mathbb{D}[i]$. A natural number k is a *denominator exponent* for t if $\gamma^k t \in \mathbb{Z}[i]$, equivalently, if $t = s/\gamma^k$ for some $s \in \mathbb{Z}[i]$. The smallest such k is the *least denominator exponent* of t , denoted $\text{lde}(t)$.

We extend this notion to vectors and matrices: k is a denominator exponent for a vector or matrix if it is a denominator exponent for all of its entries, and the lde of a vector or matrix is the least such k .

Remark II.2. By direct enumeration, the lde of any two-qubit Clifford operator is at most 2.

B. Residue rings

The quotient ring admits the following well-known characterization:

$$\mathbb{Z}[i]/(\gamma^n) = \left\{ \sum_{i=0}^{n-1} b_i \gamma^i \text{ mod } (\gamma^n) \mid b_i \in \{0, 1\} \right\}.$$

We give an elementary proof by induction on n ; a similar argument for the case $\mathbb{Z}[\sqrt{2}]/(\sqrt{2}^n)$ appears in Clement [8]. The base case $n = 1$ is immediate. Assume the claim for n : every $x \in \mathbb{Z}[i]$ has the form

$$x = \sum_{i=0}^{n-1} b_i \gamma^i + k \gamma^n \text{ for some } k \in \mathbb{Z}[i].$$

Writing $k = b_n + k' \gamma$ with $b_n \in \{0, 1\}$ and substituting yields

$$x = \sum_{i=0}^n b_i \gamma^i + k' \gamma^{n+1}, \quad \text{i.e., } x \equiv \sum_{i=0}^n b_i \gamma^i \text{ mod } (\gamma^{n+1}).$$

For brevity, we omit $\text{mod } (\gamma^n)$ when context is clear, and we write the binary string $(b_0 b_1 \dots)_2$ for the sum; the parentheses and subscript may be dropped when context makes the binary-residue interpretation unambiguous.

Moreover, distinct binary strings yield distinct residue classes: suppose $\sum_{i=0}^{n-1} b_i \gamma^i \equiv \sum_{i=0}^{n-1} b'_i \gamma^i \text{ (mod } \gamma^n)$, i.e.,

$\sum_{i=0}^{n-1} (b_i - b'_i) \gamma^i = k \gamma^n$ for some $k \in \mathbb{Z}[i]$. Reducing modulo (γ) gives $b_0 = b'_0$. After cancelling $b_0 - b'_0 = 0$ and dividing through by γ , we obtain

$$\sum_{i=1}^{n-1} (b_i - b'_i) \gamma^{i-1} = k \gamma^{n-1},$$

and reducing modulo (γ) again gives $b_1 = b'_1$. Iterating shows $b_i = b'_i$ for all i .

Let $\rho_n(x)$ denote the canonical projection from $\mathbb{Z}[i]$ to its n -digit residue. The proof above shows that ρ_n is *stable* under increase of n : the first n digits of $\rho_{n+1}(x)$ coincide with $\rho_n(x)$. Moreover, ρ_2 is stable under complex conjugation: $\rho_2(x^\dagger) = \rho_2(x)$. We use $x \equiv_{\gamma^n} y$ as shorthand for $x \equiv y \text{ (mod } \gamma^n)$. Modulo γ^3 , we have

x	$\rho_3(x)$	x	$\rho_3(x)$	x	$\rho_3(x)$	x	$\rho_3(x)$
1	100	-1	101	i	111	$-i$	110
0	000	$1+i$	010	$1-i$	011	2	001

The $(\text{mod } \gamma^n)$ residue classes form a commutative ring. Modulo γ^2 , the ring addition coincides with bit-wise exclusive or (denoted $\oplus$); $(10)_2$ is the multiplicative identity; $(00)_2$ is the zero element; and multiplication by $(01)_2$ is the right shift operation. By distributivity, $11 \cdot b_0 b_1 = 10 \cdot b_0 b_1 + 01 \cdot b_0 b_1 = b_0(b_1 \oplus b_0)$. The additive inverse of an element is itself. Since $\rho_2(i) = 11$, whenever $\rho_1(x) = 1$, either $\rho_2(x) = 10$ or $\rho_2(ix) = 10$. Similarly, modulo γ^3 , the addition rule is

$$b_0 b_1 b_2 + b'_0 b'_1 b'_2 = (b_0 \oplus b'_0)(b_1 \oplus b'_1)(b_2 \oplus b'_2 \oplus (b_0 \cdot b'_0)),$$

where $\cdot$ denotes multiplication in $\mathbb{Z}[i]/(\gamma)$, equivalently the logical “and” of two bits. The multiplicative identity is 100 and the zero element is 000. Multiplication by 010 is a right shift by 1 digit and multiplication by 001 is a right shift by 2 digits. The rest of the multiplication table follows by distributivity. The additive inverse is

$$-(b_0 b_1 b_2) = b_0 b_1 (b_2 \oplus b_0).$$

From the ρ_3 table, ρ_3 is not stable under complex conjugation: if $\rho_3(x) = abc$, then $\rho_3(x^\dagger) = ab(c \oplus b)$. Since $\rho_3(i) = 111$, whenever $\rho_1(x) = 1$ there exists $k \in \{0, 1, 2, 3\}$ with $\rho_3(i^k x) = 100$.

Definition II.3. Let l be a denominator exponent for $x \in \mathbb{D}[i]$ (not required to be the least). The (n, l) -*residue* of x is $\rho_n^l(x) := \rho_n(\gamma^l x)$. For a vector or matrix A , $\rho_n^l(A)$ is defined entry-wise whenever l is a denominator exponent for A .

Definition II.4. For $x \in \mathbb{Z}[i]$, if $\rho_1(x) = 0$, we say x is *even*, otherwise *odd*.

For example, $1 + 2i$ is odd, while $1 + 3i$ is even. In our convention, an *even* Gaussian integer is one divisible by γ — not necessarily by 2. For ordinary integers $x \in \mathbb{Z}$, however, evenness as an integer coincides with evenness as a Gaussian integer, so our definition extends the standard notion of parity. The following lemmas are straightforward.

Lemma II.5. Let $\alpha = a + bi \in \mathbb{Z}[i]$. Then α is odd if and only if $\|\alpha\|^2 = a^2 + b^2$ is an odd integer.

Lemma II.6. If $\alpha, \beta \in \mathbb{Z}[i]$ satisfy $\alpha\gamma \equiv_{\gamma^{n+1}} \beta\gamma$, then $\alpha \equiv_{\gamma^n} \beta$.

Lemma II.7. For $t \in \mathbb{D}[i]$, if $k = \text{lde}(t) > 0$, then $\gamma^k t$ is odd.

Lemma II.8. The lde function on $\mathbb{D}[i]$ and matrices with $\mathbb{D}[i]$ entries is subadditive: for $x, y \in \mathbb{D}[i]$ and matrices A, B with $\mathbb{D}[i]$ entries,

$$\text{lde}(xy) \leq \text{lde}(x) + \text{lde}(y), \text{lde}(AB) \leq \text{lde}(A) + \text{lde}(B).$$

If x, y are both odd, then equality holds.

C. Action on residues

We now describe how the basic operations — multiplication by i , shifts, addition, subtraction, and the K gate — act at the level of residues. These computations underpin the case analysis in Section IV.

If $\rho_2(x) = b_0 b_1$, then $\rho_2(ix) = b_0(b_1 \oplus b_0)$; we say i acts on $b_0 b_1$ by sending it to $b_0(b_1 \oplus b_0)$. Given an n -digit residue $b_0 b_1 \dots b_{n-1}$, a *right shift* produces the $(n+1)$ -digit residue $0b_0 b_1 \dots b_{n-1}$; given an $(n+1)$ -digit residue $0b_0 b_1 \dots b_{n-1}$, a *left shift* produces $b_0 b_1 \dots b_n$. We only consider shifts that are *well-defined*: no information is lost and no unknown digits are introduced. (The usual right-shift was used when describing multiplication in $\mathbb{Z}[i]/(\gamma^2)$ and $\mathbb{Z}[i]/(\gamma^3)$.) We write $RS(x)$ for the right shift of x and $LS(x)$ for the left shift.

Lemma II.9. For $t \in \mathbb{D}[i]$, l a denominator exponent of t , we have

$$\rho_{n+1}^{l+1}(t) = \rho_{n+1}^l(\gamma t) = RS(\rho_n^l(t)),$$

and if $\gamma^l t$ is even and $l > 0$,

$$\rho_n^{l-1}(t) = \rho_n^l(t/\gamma) = LS(\rho_{n+1}^l(t)).$$

Let $x, y \in \mathbb{D}[i]$ have $\text{lde } l > 0$, so $\rho_2^l(x) = 1b$ and $\rho_2^l(y) = 1b'$ for some $b, b' \in \{0, 1\}$. Then

$$\rho_2^l(x + y) = 1b + 1b' = 0(b \oplus b'),$$

and applying a left shift gives $\rho_1^{l-1}(x + y) = b \oplus b'$. The case of $x - y$ is analogous, since subtraction in $\mathbb{Z}/(2)$ coincides with $\oplus$ (every element is its own additive inverse). Consequently, if $b \oplus b' = 1$ then $\text{lde}(x \pm y)$ decreases by 1, and if $b \oplus b' = 0$ with $l \geq 2$, then $\text{lde}(x \pm y)$ decreases by at least 2.

Now suppose further that $\rho_3^l(x) = 10b$ and $\rho_3^l(y) = 10b'$. Since dividing by γ increases the lde by 1, we obtain

$$\begin{aligned} \rho_2^l(K \begin{bmatrix} x \\ y \end{bmatrix}) &= \rho_2^l \left(\begin{bmatrix} (x+y)/\gamma \\ (x-y)/\gamma \end{bmatrix} \right) = LS(\rho_3^l \left(\begin{bmatrix} x+y \\ x-y \end{bmatrix} \right)) \\ &= \begin{bmatrix} 0(b \oplus b' \oplus 1) \\ 0(b \oplus b') \end{bmatrix}. \end{aligned}$$

Hence $\text{lde}(K \begin{bmatrix} x \\ y \end{bmatrix}) = l - 1$, with one entry having lde $l - 1$ and the other at most $l - 2$ (provided $l \geq 2$).

Remark II.10. The K gate, acting on a pair of entries with the same lde $l > 0$ and proper residues, decreases the lde of the pair by at most 1. Moreover, K is the only gate in the Clifford+CS gate set that affects the lde at all. These two facts are needed for the optimality proof.

In general, suppose $x, y \in \mathbb{D}[i]$ have lde $l > 0$. If $\rho_3^l(x) = 1b_0 b_1$ and $\rho_3^l(y) = 1b'_0 b'_1$, then

$$\rho_2^l(K \begin{bmatrix} x \\ y \end{bmatrix}) = \begin{bmatrix} (b_0 \oplus b'_0)(b_1 \oplus b'_1 \oplus 1) \\ (b_0 \oplus b'_0)(b_1 \oplus b'_1) \end{bmatrix}.$$

If $\rho_3^l(x) = 0b_0 b_1$ and $\rho_3^l(y) = 0b'_0 b'_1$, then

$$\rho_2^l(K \begin{bmatrix} x \\ y \end{bmatrix}) = \begin{bmatrix} (b_0 \oplus b'_0)(b_1 \oplus b'_1) \\ (b_0 \oplus b'_0)(b_1 \oplus b'_1) \end{bmatrix}.$$

If $\rho_3^l(x) = 1b_0 b_1$ and $\rho_3^l(y) = 0b'_0 b'_1$, then

$$\rho_3^{l+1}(K \begin{bmatrix} x \\ y \end{bmatrix}) = \begin{bmatrix} 1(b_0 \oplus b'_0)(b_1 \oplus b'_1) \\ 1(b_0 \oplus b'_0)(b_1 \oplus b'_1) \end{bmatrix}.$$

If $\rho_3^l(x) = 0b_0 b_1$ and $\rho_3^l(y) = 1b'_0 b'_1$, then

$$\rho_3^{l+1}(K \begin{bmatrix} x \\ y \end{bmatrix}) = \begin{bmatrix} 1(b_0 \oplus b'_0)(b_1 \oplus b'_1) \\ 1(b_0 \oplus b'_0)(b_1 \oplus b'_1 \oplus 1) \end{bmatrix}.$$

III. SPECIAL UNITARIES

With the residue toolkit in place, we now turn to three families of Clifford+CS operators that play distinguished roles in the synthesis algorithm: permutation matrices, diagonal unitary matrices, and the generalized permutations that combine them. The latter are precisely the lde-0 operators, and they serve as the “zero step” of the algorithm in Section IV.

A. Permutation matrices

Let $a_0 a_1 a_2 a_3$ be a permutation of $\{0, 1, 2, 3\}$. The permutation matrix $P_{a_0 a_1 a_2 a_3}$ maps the standard basis vector e_i to e_{a_i} . All 24 permutation matrices are Clifford operators, and each can be implemented by a circuit of length at most 3; see Table I.

B. Diagonal unitary matrices

Every diagonal unitary 4×4 matrix with entries in $\mathbb{D}[i]$ has the form

$$\text{diag}(i^a, i^b, i^c, i^d), \quad a, b, c, d \in \{0, 1, 2, 3\}.$$

P	circuit	P	circuit	P	circuit
0123	empty	1023	$X_1 \cdot CX$	2103	$X_0 \cdot XC$
1203	$Ex \cdot X_0 \cdot XC$	2013	$Ex \cdot X_1 \cdot CX$	0213	Ex
3210	$X_1 \cdot X_0$	2310	$X_0 \cdot CX$	2130	$Ex \cdot X_1 \cdot XC$
3120	$Ex \cdot X_1 \cdot X_0$	1320	$Ex \cdot X_0 \cdot CX$	1230	$X_1 \cdot XC$
3012	$XC \cdot X_1$	0312	$Ex \cdot XC$	0132	CX
3102	$Ex \cdot CX \cdot X_0$	1302	$Ex \cdot X_0$	1032	X_1
3021	$Ex \cdot XC \cdot X_1$	0321	XC	0231	$Ex \cdot CX$
3201	$CX \cdot X_0$	2301	X_0	2031	$Ex \cdot X_1$

TABLE I. Circuit implementations of the 24 permutation matrices.

Each such matrix is implementable by a Clifford+CS circuit of length at most 6, up to a global phase i^l , using at most 1 CS gate. Indeed, the $2^6 \cdot 4 = 4^4$ products

$$Z_0^{b_0} Z_1^{b_1} S_0^{b_2} S_1^{b_3} CZ^{b_4} CS^{b_5} \cdot i^k, \quad b_i \in \{0, 1\}, \quad k \in \{0, 1, 2, 3\},$$

are all distinct and diagonal — exhausting the 4^4 diagonal unitaries.

C. Generalized permutation unitary matrices

Combining the previous two families gives the generalized permutations, the building blocks of every lde-0 Clifford+CS operator.

A *generalized permutation* is a unitary matrix over $\mathbb{D}[i]$ having the same nonzero pattern as a permutation matrix, but with nonzero entries equal to arbitrary powers of i . Every 4×4 generalized permutation factors as a product of a permutation matrix and a diagonal unitary matrix; consequently, it is implementable by a circuit of length at most 9 using at most 1 CS gate and no K gate. The group of generalized permutations is the semidirect product of the permutation group and the diagonal unitary group. Concretely, for any permutation matrix P and diagonal matrix D , there is a diagonal matrix D' with

$$PD = D'P,$$

and D, D' have the same CS-count (see Definition V.1).

There are finitely many generalized permutation Clifford+CS operators, each of which has lde 0. Conversely, every lde-0 Clifford+CS operator is a generalized permutation, because a unit vector with lde 0 must have the form $(i^k, 0, 0, 0)^T$ for some $k \in \{0, 1, 2, 3\}$, up to a permutation of its entries.

IV. THE EXACT SYNTHESIS ALGORITHM

The method in this section is a residue analysis over the ring $\mathbb{Z}[\frac{1}{2}, i]$, similar in spirit to Russell's [34] analysis

over $\mathbb{Z}[\frac{1}{\sqrt{2}}, i]$. Such techniques are widely used in exact synthesis; see, e.g., [3, 13, 14, 17, 29]. Whereas prior work uses residues in an ad hoc manner, we develop in Section II a uniform treatment based on binary residues, which we conjecture generalizes to any ξ -ring in the sense of [27].

The main idea of the synthesis algorithm is induction on the lde of the input. The key step reduces the lde by 1, achieved through a residue analysis of the current operator.

A. Residue pattern

We begin by classifying, up to row and column permutations, the $(1, l)$ -residues that can occur for an operator $A \in CCS$ with lde l . The classification, given in Lemma IV.1 below, yields six patterns; subsequent analysis is carried out one pattern at a time.

We begin with several lemmas. Since $\mathbb{Z}[i]$ and $\mathbb{D}[i]$ are subrings of $\mathbb{C}$, the standard notions of complex conjugation (denoted $\cdot^\dagger$), inner product, norm, transposition (denoted $\cdot^T$), and conjugate transposition (also denoted $\cdot^\dagger$) carry over. For $v \in \mathbb{D}[i]^n$, the *norm* is $\|v\| = \sqrt{v^\dagger v}$; since $v^\dagger v \in \mathbb{D}[i] \cap \mathbb{R}$, we have $\|v\|^2 = v^\dagger v \in \mathbb{D}$. Similarly, $v^\dagger v \in \mathbb{Z}$ when $v \in \mathbb{Z}[i]^n$. A *unit vector* is a vector of norm 1. We write v_j for the j th entry of v , indexing from 0.

Lemma IV.1. *For any $A \in CCS$ with lde l , there exist permutation matrices L and R such that $\rho_1^l(LAR)$ or $\rho_1^l(LAR)^T$ equals one of the following*

$$\begin{aligned}
(i) \quad & \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad (ii) \quad \begin{bmatrix} 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \quad (iii) \quad \begin{bmatrix} 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix}, \\
(iv) \quad & \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \quad (v) \quad \begin{bmatrix} 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}, \quad (vi) \quad \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}.
\end{aligned}$$

We say A has such a pattern and write $\text{pat}(A)$ for the pattern of A . Moreover, A has pattern (i) if and only if $\text{lde}(A) = 0$, i.e., when A is a generalized permutation. Patterns (iv), (v), and (vi) occur only when $\text{lde}(A) > 1$. Each Clifford operator has pattern (i), (iii), or (vi).

Note that all cases except Case (iv) are transposition-invariant up to permutation of rows and columns, e.g., $(vi)^T = (vi)$. We prove the lemma after establishing several auxiliary results.

Lemma IV.2. *Let v be a unit vector in $\mathbb{D}[i]^4$ with lde $l > 0$. There exists a generalized permutation matrix P such that $\rho_1^l(Pv) = (1, 1, 0, 0)^T$ or $(1, 1, 1, 1)^T$, $\rho_2^l(Pv) = (10, 10, 01, 00)^T$ or $(10, 10, 10, 10)^T$ when $l > 1$, and $\rho_2^l(Pv) = (10, 10, 00, 00)^T$ when $l = 1$.*

Proof. Let $w = \gamma^l v \in \mathbb{Z}[i]^4$. We first show that w has an even number of odd entries. Since

$$\sum_{j=0}^3 \|w_j\|^2 = \|w\|^2 = |\gamma|^{2l} \|v\|^2 = (\gamma^\dagger \gamma)^l \equiv 0 \pmod{\gamma},$$

an even number of indices $j \in \{0, \dots, 3\}$ have $\|w_j\|^2$ odd. By Lemma II.5, an even number of the w_j are odd.

We can then find a permutation matrix P such that $\rho_2^l(Pv) = (1a, 1b, 0c, 0d)^T$ or $(1a, 1b, 1c, 1d)^T$. In the latter case, multiplying by a suitable diagonal matrix (i.e., scaling individual entries by appropriate powers of i) makes all second digits zero, yielding $(10, 10, 10, 10)^T$.

In the former case, multiplying by a suitable diagonal matrix makes the second digits of the odd entries zero, yielding $(10, 10, 0c, 0d)^T$. Applying X_0CKX_0 — equivalently, applying K to the pair of odd entries — produces $(00, 01, 0c, 0d)^T$ or $(01, 00, 0c, 0d)^T$. Since multiplication by unitaries preserves unit vectors, this is the residue of a unit vector with lde $l-1$. When $l > 1$, its $(1, l-1)$ -residue must equal $(1, 1, 0, 0)^T$ up to a permutation, so $c = 1$ and $d = 0$ up to a permutation.

When $l = 1$, the only unit vectors are $\frac{1}{\gamma}(i^a, i^b, 0, 0)^T$ for $a, b \in \{0, 1, 2, 3\}$, up to a permutation. $\square$

Lemma IV.3. *Let u, v be unit vectors in $\mathbb{D}[i]^4$ with lde $l > 0$ such that $u \perp v$, i.e., $u^\dagger v = 0$. Then there exists a generalized permutation matrix Q such that $\rho_1^l(Q[u|v])$ equals one of*

$$\begin{bmatrix} 1 & 1 \\ 1 & 1 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}, \begin{bmatrix} 1 & 0 \\ 1 & 0 \\ 0 & 1 \\ 0 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 1 \\ 1 & 1 \\ 1 & 0 \\ 1 & 0 \end{bmatrix}, \begin{bmatrix} 1 & 1 \\ 1 & 1 \\ 0 & 1 \\ 0 & 1 \end{bmatrix}, \text{ or } \begin{bmatrix} 1 & 1 \\ 1 & 1 \\ 1 & 1 \\ 1 & 1 \end{bmatrix}.$$

Proof. By Lemma IV.2, each column can have only an even number of 1s. Up to a permutation of rows, the only missing case is

$$\begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 0 & 1 \\ 0 & 0 \end{bmatrix}.$$

Then $\rho_1^l(u^\dagger v) = \sum_{j=0}^3 \rho_1^l(u_j^\dagger v_j) = 1$, contradicting $\rho_1^l(u^\dagger v) = \rho_1^l(0) = 0$. In other words, the number of indices at which both u and v have $(1, l)$ -residue equal to 1 must be even. $\square$

Proof of Lemma IV.1. The rows and columns of a unitary matrix are unit vectors, and all row (column) vectors are pairwise orthogonal. Lemmas IV.2 and IV.3 force the pattern to be (ii)–(vi) when $l > 0$. When $l = 0$, the only unitaries with lde 0 are the generalized permutations, which have pattern (i). When $l = 1$, the only unit vectors are $\frac{1}{\gamma}(i^a, i^b, 0, 0)^T$ up to a permutation, so the residue cannot have four 1s in a column (row). Hence

patterns (iv), (v), and (vi) occur only when $\text{lde}(A) > 1$. By enumeration, each Clifford operator has pattern (i), (iii), or (vi). $\square$

B. Transitions between patterns

Lemma IV.1 classifies the residue patterns of an arbitrary $A \in \mathcal{CCS}$. We now show that, for each non-identity pattern, a short prefix or suffix of K_1 gates and generalized permutations strictly decreases the lde, eventually reaching pattern (i).

Lemma IV.4. *For any $A \in \mathcal{CCS}$ with lde $l > 0$, there exist generalized permutations L, R, L', R' such that one of the following holds:*

- $\text{lde}(K_1 L A R) \leq l - 1$;
- $\text{lde}(L A R K_1) \leq l - 1$;
- $\text{lde}(L' K_1 L A R R' K_1) \leq l - 1$;
- $\text{lde}(K_1 L' K_1 L A R R') \leq l - 1$;
- $\text{lde}(L' C K L A R) \leq l - 1$ (only when $\text{lde}(A) = 1$).

Each inequality is in fact an equality: by Remark II.10, K acting on a pair of entries decreases the lde by at most 1. (For the cases with two K gates, only the outer K decreases the lde; see the proof.) The proof proceeds by case analysis on $\rho_1^l(A)$. Since $l > 0$, Case (i) is excluded, so we begin with Case (ii).

1. Case ii

If $l > 1$, by Lemma IV.2, up to left and right multiplication by generalized permutations, $\text{pat}(A)$ refines to

$$\begin{bmatrix} 10 & 10 & 01 & 00 \\ 10 & 1a & 0b & 0\bar{b} \\ 01 & 0c & 0d & 0e \\ 00 & 0\bar{c} & 0f & 0g \end{bmatrix},$$

where $\bar{b} = b \oplus 1$ and a, b, c, d, e, f, g are indeterminates. We use indeterminates in residue matrices throughout this section without explicit declaration. Taking the inner product of the first two columns gives

$$10 + 1a + 00 + 00 = 00 \implies a = 0.$$

The inner product of the second and third rows gives $c = 1$, and similarly $b = 1$. Now $\text{pat}(A)$ refines to

$$\begin{bmatrix} 10 & 10 & 01 & 00 \\ 10 & 10 & 01 & 00 \\ 01 & 01 & 0d & 0e \\ 00 & 00 & 0f & 0g \end{bmatrix}.$$

Moreover, since $l > 1$ and the third and fourth rows (columns) have lde $l - 1 > 0$, Lemma IV.2 gives $d = f$, $d = e$, and $e = g$, so $d = e = f = g$. Now $\text{pat}(A)$ refines to

$$\begin{bmatrix} 10 & 10 & 01 & 00 \\ 10 & 10 & 01 & 00 \\ 01 & 01 & 0d & 0d \\ 00 & 00 & 0d & 0d \end{bmatrix}.$$

Applying $K_1 S_1$ on the left reduces this to Case (iv).

If $l = 1$, up to left and right multiplication by generalized permutations, $\text{pat}(A)$ refines to

$$\begin{bmatrix} 10 & 10 & 00 & 00 \\ 10 & 1a & 00 & 00 \\ 00 & 00 & 00 & 00 \\ 00 & 00 & 00 & 00 \end{bmatrix}.$$

As before, $a = 0$. Applying $X_0 \cdot CK \cdot X_0$ on the left decreases the lde to 0, reaching pattern (i).

Remark IV.5. The two generalized permutations used to refine the pattern can be chosen to be CS-free. The $(2, l)$ -residues of the two rows (columns) with even entries are stable under multiplication by i , so each generalized permutation may be updated by a factor of CS without affecting its purpose. Moreover, if a generalized permutation P is implementable using one CS gate, then $P \cdot CS$ is implementable using none.

2. Case iii

If $l > 1$, by Lemma IV.2, up to left and right multiplication by generalized permutations, $\text{pat}(A)$ refines to

$$\begin{bmatrix} 10 & 10 & 01 & 00 \\ 10 & 1a & 0b & 0\bar{b} \\ 01 & 0c & 10 & 10 \\ 00 & 0\bar{c} & 10 & 1d \end{bmatrix}.$$

The inner product of the first two columns gives $a = 0$; similarly $d = 0$. The inner product of the second and third rows gives $c = 0$, and similarly $b = 0$. Now $\text{pat}(A)$ refines to

$$\begin{bmatrix} 10 & 10 & 01 & 00 \\ 10 & 10 & 00 & 01 \\ 01 & 00 & 10 & 10 \\ 00 & 01 & 10 & 10 \end{bmatrix}.$$

Applying $K_1 S_1$ on the left reduces this to Case (vi).

If $l = 1$, by Lemma IV.2, up to left and right multiplication by generalized permutations, $\text{pat}(A)$ refines to

$$\begin{bmatrix} 10 & 10 & 00 & 00 \\ 10 & 1a & 00 & 00 \\ 00 & 00 & 10 & 10 \\ 00 & 00 & 10 & 1d \end{bmatrix}.$$

As before, $a = 0$ and $d = 0$. Applying K_1 on the left decreases the lde to 0, reaching pattern (i).

3. Case iv

This case occurs only when $l > 1$. By Lemma IV.2, up to left and right multiplication by generalized permutations, $\text{pat}(A)$ refines to

$$\begin{bmatrix} 10 & 10 & 10 & 10 \\ 10 & 1a & 1b & 1c \\ 01 & 0d & 0e & 0f \\ 00 & 0\bar{d} & 0\bar{e} & 0\bar{f} \end{bmatrix}.$$

The inner product of the first two columns gives $a = 0$, and similarly $b = c = 0$. Since the third row has lde $l - 1 > 0$, either $d = e = f = 1$ or exactly one of d, e, f is 1 and the rest are 0. In the latter case, up to a column permutation, we may assume $d = 1$ and $e = f = 0$. In both cases, right multiplication by K_1 decreases the lde, and moreover, up to permutations of rows and columns, reaches the $(1, l - 1)$ -residue

$$\begin{bmatrix} 1 & 0 & 1 & 0 \\ p & \bar{p} & q & \bar{q} \\ x & x & y & y \\ z & z & w & w \end{bmatrix}.$$

Since $l - 1 > 0$, taking the inner product of the first two columns gives $x = z$, and similarly $y = w$. Then $p = 1$ since each column (row) has an even number of odd entries; similarly, $q = 1$. Taking the inner product of the first and third rows gives $x = y$, and the inner product of the first two rows gives $p = q$. Depending on whether $x = 0$ or 1, the target pattern is (ii) or (v).

Remark IV.6. As in Remark IV.5, the left generalized permutation can be chosen to be CS-free.

4. Case v

This case occurs only when $l > 1$. By Lemma IV.2, up to left and right multiplication by generalized permutations, $\text{pat}(A)$ refines to

$$\begin{bmatrix} 10 & 10 & 01 & 00 \\ 10 & 1a & 00 & 01 \\ 10 & 1b & 10 & 1f \\ 10 & 1c & 1e & 10 \end{bmatrix}.$$

The inner product of the first two rows gives $a = 0$. The inner product of the first and third rows gives $b = 1$, and similarly $c = 1$. The inner product of the second and third columns gives $e = 1$, and similarly $f = 1$. Now the

pattern is

$$\begin{bmatrix} 10 & 10 & 01 & 00 \\ 10 & 10 & 00 & 01 \\ 10 & 11 & 10 & 11 \\ 10 & 11 & 11 & 10 \end{bmatrix}.$$

Applying K_1 on the left reduces this to Case (iv).

5. Case vi

This case occurs only when $l > 1$. By Lemma IV.2, up to left and right multiplication by generalized permutations, $\text{pat}(A)$ refines to

$$\begin{bmatrix} 10 & 10 & 10 & 10 \\ 10 & 1a & 1d & 1g \\ 10 & 1b & 1e & 1h \\ 10 & 1c & 1f & 1j \end{bmatrix}.$$

The inner product of the first two columns shows that a, b, c cannot all be 1. Up to a row permutation, we may assume $a = 0$, which gives $b = c$, and similarly $d = g$.

If $b = c = 1$ and $d = g = 1$, then $\text{pat}(A)$ is

$$\begin{bmatrix} 10 & 10 & 10 & 10 \\ 10 & 10 & 11 & 11 \\ 10 & 11 & 1e & 1h \\ 10 & 11 & 1f & 1j \end{bmatrix}.$$

The inner product of the first and third columns gives $f = \bar{e}$, and similarly $h = \bar{e}$ and $j = \bar{f}$. Now $\text{pat}(A)$ is

$$\begin{bmatrix} 10 & 10 & 10 & 10 \\ 10 & 10 & 11 & 11 \\ 10 & 11 & 1e & 1\bar{e} \\ 10 & 11 & 1\bar{e} & 1e \end{bmatrix}.$$

We show this pattern is impossible by examining the $(3, l)$ -residue. Up to multiplication by powers of -1 on appropriate rows and columns (achieved by multiplying by suitable diagonal matrices on the left and right), the $(3, l)$ -residue refines to

$$\begin{bmatrix} 100 & 100 & 100 & 100 \\ 100 & 10x & 11u & 11p \\ 100 & 11y & 1ev & 1\bar{e}q \\ 100 & 11z & 1\bar{e}w & 1er \end{bmatrix}.$$

The inner products of the first column with the second and third columns give

$$\begin{aligned} 100 + 10x + 11y + 11z &= 000 \implies x \oplus y \oplus z = 0, \\ 100 + 11u + 1ev + 1\bar{e}w &= 000 \implies u \oplus v \oplus w = 0. \end{aligned}$$

The inner product of the second and third columns gives

$$100 + 11(u \oplus x) + (1ev + 01e + 00\bar{y}) + (1\bar{e}w + 01\bar{e} + 00\bar{z}) = 000 \implies x \oplus y \oplus z \oplus u \oplus v \oplus w = 1, \text{ a contradiction.}$$

If $b = c = 0$, the inner product of the first and third rows gives $e = h$; similarly, $f = j$. So $\text{pat}(A)$ refines to

$$\begin{bmatrix} 10 & 10 & 10 & 10 \\ 10 & 10 & 1d & 1d \\ 10 & 10 & 1e & 1e \\ 10 & 10 & 1f & 1f \end{bmatrix}.$$

The inner product of the first and third columns gives $d \oplus e \oplus f = 0$, so at least one of d, e, f is 0. Up to a row permutation, we may assume $d = 0$. So, $\text{pat}(A)$ refines to

$$\begin{bmatrix} 10 & 10 & 10 & 10 \\ 10 & 10 & 10 & 10 \\ 10 & 10 & 1e & 1e \\ 10 & 10 & 1f & 1f \end{bmatrix}.$$

The inner product of the first and third columns implies $e = f$, so left multiplication by K_1 reduces the lde, and moreover, up to permutations of rows and columns, reaching $(1, l-1)$ -residue

$$\begin{bmatrix} 1 & x & y & z \\ 0 & \bar{x} & \bar{y} & \bar{z} \\ 1 & u & v & w \\ 0 & \bar{u} & \bar{v} & \bar{w} \end{bmatrix}.$$

Taking the inner product of the first two columns implies $x = u$, and similarly $y = v$ and $z = w$. Depending on whether x, y, z are all 1 or not, the target pattern is (iv) or (iii).

The case $d = g = 0$ is similar.

6. Case (iv)^T

This case is analogous to Case (iv), but the lde is decreased by a right multiplication by K_1 rather than a left multiplication.

Thus we have exhausted all cases. The transitions are summarized in Figure 1.

A path that starts from any case and ends at (i) is called *complete*; following such a path drives the lde of the starting point down to 0. For $A \in \text{CCS}$, we define $\text{prkc}(A)$ to be the number of K gates along the complete path starting from A .

The next lemma shows that $\text{prkc}(A)$ is determined by the starting pattern and lde alone.

Lemma IV.7 (prkc of a complete path). *For $A \in \text{CCS}$ with lde $l > 0$, $\text{prkc}(A)$ is determined by l and $\rho_1^l(A)$ as shown in Table II.*

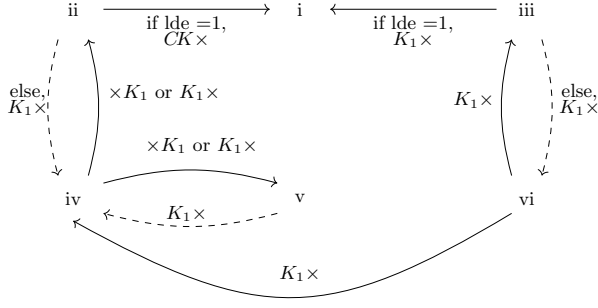


FIG. 1. Transitions between cases. The annotation $K_1 \times$ ($\times K_1$) on an arrow means that if A has the pattern at the tail, then $K_1 LAR$ ($LARK_1$) has the pattern at the head for suitably chosen generalized permutations L and R , and similarly for the other annotations. Dashed arrows preserve the lde; the rest decrease it by 1. Note that CK decomposes into a circuit with two K gates as in Equation (1).

case	prkc	typical cases
(ii)	$2l$	$ii \rightarrow i, ii \rightarrow iv \rightarrow ii \rightarrow i, ii \rightarrow iv \rightarrow v \rightarrow iv \rightarrow ii \rightarrow i$
(iii)	$2l - 1$	$iii \rightarrow i, iii \rightarrow vi \rightarrow iii \rightarrow i$
(iv)	$2l - 1$	$iv \rightarrow ii \rightarrow i, iv \rightarrow ii \rightarrow iv \rightarrow ii \rightarrow i, iv \rightarrow v \rightarrow iv \rightarrow ii \rightarrow i$
(v)	$2l$	$v \rightarrow iv \rightarrow ii \rightarrow i, v \rightarrow iv \rightarrow v \rightarrow iv \rightarrow ii \rightarrow i$
(vi)	$2l - 2$	$vi \rightarrow iii \rightarrow i, vi \rightarrow iv \rightarrow ii \rightarrow i, vi \rightarrow iii \rightarrow vi \rightarrow iii \rightarrow i$

TABLE II. prkc of a complete path.

Proof. By following the paths. $\square$

Corollary IV.8 (prkc together with residue determines lde). *Let $A \in CCS$ and $p = \text{prkc}(A)$. If p is odd, then $\text{lde}(A) = (p+1)/2$. If p is even, then $\text{lde}(A) = p/2$ when $\rho_1^l(A) \neq vi$, and $\text{lde}(A) = p/2 + 1$ when $\rho_1^l(A) = vi$.*

We are now ready to bound the optimal K -count of A by $\text{prkc}(A)$.

Lemma IV.9. *For $A \in CCS$, $\text{prkc}(A) \geq \text{kc}(A)$, where $\text{kc}(A)$ is the optimal K -count of Definition V.1.*

Proof. We construct a circuit C with raw K -count $\text{prkc}(A)$ that implements A . If $\text{lde}(A) = 0$, then A is a generalized permutation, and Section III C gives a circuit using zero K gates that implements A . If $\text{lde}(A) > 0$, then by Lemma IV.4 and Lemma IV.7, we have a complete path with K -count $\text{prkc}(A)$ that drives $\text{lde}(A)$ to 0. That is, after merging adjacent generalized permutations as needed, there exist generalized permutations $L_1, \dots, L_p, R_1, \dots, R_q$ with

$$\text{lde}(L_p K_1 \cdots L_2 K_1 L_1 A R_1 K_1 R_2 K_1 \cdots R_q) = 0;$$

that is, there exists a generalized permutation P such that

$$L_p K_1 \cdots L_2 K_1 L_1 A R_1 K_1 R_2 K_1 \cdots R_q = P.$$

Therefore,

$$A = L_1^{-1} i K_1 L_2^{-1} i K_1 \cdots L_p^{-1} P R_q^{-1} i K_1 \cdots R_2^{-1} i K_1 R_1^{-1}.$$

Let C be the right-hand side with each generalized permutation $L_j^{-1} i$, $R_j^{-1} i$, and $L_p^{-1} P R_q^{-1}$ replaced by a circuit implementing it as in Section III C. Then $\text{rkc}(C) = \text{prkc}(A)$. $\square$

The proof of Lemma IV.9 yields an algorithm for the exact synthesis of a two-qubit Clifford+CS operator A with raw K -count $\text{prkc}(A) \in \{2l, 2l-1, 2l-2\}$, where $l = \text{lde}(A)$. We denote this function by $\text{synth} : CCS \rightarrow CIR$.

C. Complexity

Each iteration reduces the lde by 1 using only finitely many operations, and unitaries with lde 0 are generalized permutations. Hence synth runs in time $O(\text{lde}(A))$ for any input $A \in CCS$.

V. synth IS K-OPTIMAL

Lemma IV.9 shows that synth uses no more than $\text{prkc}(A)$ K gates. To establish K -optimality, we prove the matching lower bound: every Clifford+CS implementation of A uses at least $\text{prkc}(A)$ K gates. We do so by showing that the local pattern transitions and the complete path transitions in Figure 1 are themselves K -optimal. We begin by defining gate-count metrics and a notion of *descent* that captures arbitrary ways of decreasing the lde.

Definition V.1. Let D be a Clifford+CS circuit, and recall that $[D]$ is the unitary it implements. We define

- $\text{rkc}(D)$ = the number of K gates in D .
- $\text{kc}(D) = \min \{\text{rkc}(C) \mid C \in CIR \text{ and } [C] = [D]\}$.
- $\text{rcs}(D)$ = the number of CS gates in D .
- $\text{cs}(D) = \min \{\text{rcs}(C) \mid C \in CIR \text{ and } [C] = [D]\}$.
- $\text{rlen}(D)$ = the total number of gates in D .
- $\text{len}(D) = \min \{\text{rlen}(C) \mid C \in CIR \text{ and } [C] = [D]\}$.

By abuse of notation, cs , kc , and len are also defined on operators implementable by circuits. We say D is *K-optimal* if $\text{rkc}(D) = \text{kc}(D)$, *CS-optimal* if $\text{rcs}(D) = \text{cs}(D)$, and *shortest* if $\text{rlen}(D) = \text{len}(D)$. We call $\text{rkc}(D)$ and $\text{rcs}(D)$ the (raw) K -count and (raw) CS -count of D , and $\text{kc}(D)$ and $\text{cs}(D)$ the optimal K -count and optimal CS -count.

Definition V.2. An *lde-descent* (or simply *descent*) of $A \in \text{CCS}$ is a pair $L, R \in \text{CCS}$ with $\text{lde}(LAR) \leq \text{lde}(A)$. Since L and R may each be written as an alternating sequence of K_1 gates and generalized permutations (cf. Equation (3)), a descent is equivalently specified by generalized permutations $L_1, \dots, L_p, R_1, \dots, R_q$ such that

$$\text{lde}(L_p \cdots K_1 L_2 K_1 L_1 A R_1 K_1 R_2 K_1 \cdots R_q) \leq \text{lde}(A).$$

A *sub-descent* is given by $L_1, \dots, L_{p'}, R_1, \dots, R_{q'}$ for some $1 \leq p' \leq p$ and $1 \leq q' \leq q$, and we call the remaining part (including all its K gates) the *exterior*. Each path in Figure 1 describes a descent, called a *path descent*; a *complete path descent* is one whose lde reaches 0. The *K-count* of a descent is the number of K gates it uses, and a descent of minimum K-count is *K-optimal*. We extend the notation rk and kc to descents, and write *n-descent* when the lde decreases by exactly n .

The first step is to show that the smallest building block — a single lde-decreasing transition along Figure 1 — is already K-optimal.

Lemma V.3 (Path 1-descent is optimal). *Any path 1-descent for $A \in \text{CCS}$ with $\text{lde}(A) > 0$ (i.e., an lde-decreasing edge or an lde-preserving edge followed by an lde-decreasing edge in Figure 1) is K-optimal.*

Proof. Let $l = \text{lde}(A)$. The claim is trivial if $\rho_1^l(A) \in \{(iv), (vi)\}$ or $\rho_1^l(A) = (iii)$: in each case, one K gate decreases the lde by 1, and at least one is needed to change the lde at all.

For the remaining cases — $\rho_1^l(A) \in \{(ii), (v)\}$, or $\rho_1^l(A) = (iii)$ with $l > 1$ — the path uses two K gates. We claim this is optimal. The number of single- K -gate descents is finite, since in $L_2 K_1 L_1 A R_1$ or $L_1 A R_1 K_1 R_2$ the generalized permutations L_1, L_2, R_1 and R_2 each range over a finite set; enumeration confirms that none decrease the lde. Moreover, since multiplication by generalized permutations does not change the lde, we only need an enumeration of descents $K_1 L_1 A$ or $A R_1 K_1$. $\square$

Lemma V.4 (K-count 1 1-descent is pattern-locking). *The source-target pattern of any K-count-1 1-descent matches that of an undashed edge in Figure 1.*

Proof. The number of K-count-1 1-descents from each residue pattern is finite, so the claim can be checked by enumeration. We give a cleaner argument below.

For a K-count-1 1-descent from pattern S to pattern T , we show that the edge $S \rightarrow T$ appears in Figure 1. It is easier to consider the reverse: S is a 1-ascent from T . For $T \in \{(i), (ii), (iii), (iv)\}$, applying an lde-increasing K gate doubles the odd entries (at the larger lde), forcing $S = (iii), (iv), (vi), (vi)$ respectively. For $T = (v)$, the lde-increasing K gate produces eight odd entries in two rows (or columns), forcing $S = (iv)$. For $T = (vi)$, there is no K-count-1 1-ascent. $\square$

Repeated K-optimal 1-descents do not necessarily yield an overall K-optimal descent — for example, when reducing the lde from 10 to 0. Combining pattern-locking

with the fact that the lde decreases one step at a time (Remark II.10), however, we show below that every complete path descent is K-optimal: there is no shortcut to $\text{lde} = 0$.

The proof rests on two further lemmas. The first (Lemma V.5) states that the outer steps of a K-optimal descent themselves form a K-optimal descent. The second (Lemma V.6) rules out a bad case that would otherwise complicate the induction.

Lemma V.5 (The exterior of a K-optimal descent is K-optimal). *For $A \in \text{CCS}$ with lde l , if $L_p K_1 \cdots L_2 K_1 L_1 A R_1 K_1 R_2 K_1 \cdots R_q$ is a K-optimal descent, then any exterior p', q' of it is a K-optimal descent of $L_{p'} K_1 \cdots L_2 K_1 L_1 A R_1 K_1 R_2 K_1 \cdots R_{q'}$.*

Proof. Suppose not. Then some descent E of $L_{p'} K_1 \cdots L_2 K_1 L_1 A R_1 K_1 R_2 K_1 \cdots R_{q'}$ uses fewer K gates than the exterior, and replacing the exterior by E yields a descent with fewer K gates than the original, contradicting K-optimality. $\square$

Lemma V.6. *A K-count 1 0-descent from pattern vi cannot reach pattern ii or v .*

Proof. Equivalently, no K-count-1 0-descent from pattern (ii) or (v) can reach pattern (vi). As in the proof of Lemma IV.4, we may assume without loss of generality that the $(2, l)$ -residues of patterns (ii) and (v) are

$$\begin{bmatrix} 10 & 10 & 01 & 00 \\ 10 & 10 & 01 & 00 \\ 01 & 01 & 0d & 0d \\ 00 & 00 & 0d & 0d \end{bmatrix} \quad \text{and} \quad \begin{bmatrix} 10 & 10 & 01 & 00 \\ 10 & 10 & 00 & 01 \\ 10 & 11 & 10 & 11 \\ 10 & 11 & 11 & 10 \end{bmatrix}.$$

A direct check shows that no K-count-1 0-descent from these residues yields pattern (vi). Alternatively, since the K-count-1 0-descents from pattern (vi) form a finite set, the lemma can also be verified by enumeration. $\square$

Lemma V.7 (Complete path descent is optimal). *For $A \in \text{CCS}$ with lde l , the complete path descent of A , i.e., the path l -descent of A , is K-optimal.*

Proof. By induction on $\text{lde}(A)$. If $\text{lde}(A) = 0$, the empty 0-descent is K-optimal. Suppose the claim holds for all A with $\text{lde}(A) = l > 0$, and let A have $\text{lde}(A) = l + 1$. Let $D = L_p K_1 \cdots L_2 K_1 L_1 A R_1 K_1 R_2 K_1 \cdots R_q$ be a K-optimal $(l + 1)$ -descent. Among its sub-descents, choose one of minimal K-count, D_1 with indices $1 \leq p' \leq p$, $1 \leq q' \leq q$, such that $\text{lde}(M) = l$, where $M = L_{p'} K_1 \cdots L_2 K_1 L_1 A R_1 K_1 R_2 K_1 \cdots R_{q'}$. By the induction hypothesis, the path l -descent H from M is K-optimal, and Lemma V.5 gives that the exterior E of D_1 is also K-optimal, so $\text{kc}(E) = \text{rk}(E) = \text{prkc}(M)$. Since D_1 has minimal K-count, its K-count-1 exterior E_1 decreases the lde from $l + 1$ to l (otherwise E_1 could be stripped, contradicting minimality). By pattern-locking, the pattern-pair of E_1 lies in $\{iii \rightarrow i, iv \rightarrow ii, iv \rightarrow v, vi \rightarrow iii, vi \rightarrow iv\}$, so

$\rho_1^l(M) \neq vi$ and $\text{prkc}(M) \in \{2l, 2l-1\}$ by Lemma IV.7. Note $\text{rk}(D_1) + \text{rk}(E) = \text{rk}(D)$ and $\text{rk}(D) < \text{prkc}(A)$ (otherwise $\text{prkc}(A)$ is already optimal), hence $\text{rk}(D_1) < \max\{2(l+1)-2l, 2(l+1)-(2l-1)\} = 3$. Also $\text{rk}(D_1) > 0$, since at least one K gate is needed to change the lde. It remains to show that $\text{rk}(D) \geq \text{prkc}(A)$.

If $\text{rk}(D_1) = 1$, pattern-locking forces $\rho_1^{l+1}(A) \rightarrow \rho_1^l(M) \in \{iii \rightarrow i, iv \rightarrow ii, iv \rightarrow v, vi \rightarrow iii, vi \rightarrow iv\}$. In the first three subcases, $\text{prkc}(A) = 2(l+1) - 1 = 1 + 2l = \text{rk}(D_1) + \text{rk}(E) = \text{rk}(D)$; in the latter two, $\text{prkc}(A) = 2(l+1) - 2 = 1 + (2l-1) = \text{rk}(D_1) + \text{rk}(E) = \text{rk}(D)$.

If $\text{rk}(D_1) = 2$, the pattern-pair of E_1 lies in the same five-element set. In the first three subcases, $\text{rk}(D) = \text{rk}(D_1) + \text{rk}(E) = 2 + 2l = 2(l+1) \geq \text{prkc}(A)$. In the latter two, the sub-descent $D_1 \setminus E_1$ is lde-preserving with pattern-pair $\rho_1^{l+1}(A) \rightarrow vi$, so by Lemma V.6, $\rho_1^{l+1}(A) \notin \{ii, v\}$; therefore $\text{rk}(D) = \text{rk}(D_1) + \text{rk}(E) = 2 + (2l-1) = 2(l+1) - 1 \geq \text{prkc}(A)$. In every case, the path $(l+1)$ -descent of A is K -optimal. $\square$

A. Highlights

Combining the preceding lemmas yields the main K -optimality result together with bounds on the CS -count and the total gate count.

Corollary V.8. *For any $A \in CCS$, let $k = \text{prkc}(A)$. Then synth outputs a K -optimal circuit C implementing A , using k K gates, at most $k+1$ CS gates, and at most $9(k+1) + k = 10k + 9$ gates, up to a global phase.*

Proof of Corollary V.8. If synth were not K -optimal, we could synthesize A using fewer than $\text{prkc}(A)$ K gates and convert this synthesis into a descent of A with fewer K gates, contradicting the K -optimality of the path $\text{lde}(A)$ -descent.

The output is an alternating sequence of generalized permutations and K_1 gates as in Equation (3), with optimal K -count k . There are then $k+1$ generalized permutations, each implementable by at most one CS gate and at most 9 gates, up to a global phase. The claim follows. $\square$

The next theorem relates the optimal K - and CS -counts; it underlies the CS -near-optimality of synth .

Theorem V.9. *For any $A \in CCS$, $\frac{\text{kc}(A)}{2} - 1 \leq \text{cs}(A) \leq \text{kc}(A) + 1$.*

Proof. Any Clifford+ CS operator A admits a decomposition as an alternating sequence of Clifford parts C_i and CS gates,

$$A = C_0 \cdot CS \cdot C_1 \cdot CS \cdot \dots \cdot C_n. \quad (2)$$

Enumerating the two-qubit Clifford operators shows that each C_i is implementable using at most 2 K gates, so

$$\text{kc}(A) \leq 2(\text{rcs}(A) + 1) \implies \frac{\text{kc}(A)}{2} - 1 \leq \text{cs}(A).$$

Alternatively, A can be written as an alternating sequence of generalized permutations P_i and K_1 gates (replace every K_0 with $Ex \cdot K_1 \cdot Ex$):

$$A = P_0 \cdot K_1 \cdot P_1 \cdot K_1 \cdot \dots \cdot P_n. \quad (3)$$

Each generalized permutation uses at most one CS gate and no K gate, so $\text{cs}(A) \leq \text{rk}(A) + 1 \implies \text{cs}(A) \leq \text{kc}(A) + 1$. $\square$

Corollary V.10. *With $k = \text{prkc}(A)$, Theorem V.9 gives*

$$\frac{k+1}{\text{cs}(A)} \leq \frac{k+1}{\frac{k}{2}-1} = 2 + \frac{4}{k-2},$$

so the raw CS -count of the output of synth is asymptotically at most twice the optimal CS -count $\text{cs}(A)$.

We show below that the K -count of Glaudell et al.'s CS -optimal synthesis algorithm is also asymptotically at most twice optimal, suggesting that for the CS and K gates, optimality of one tends to come at the expense of the other. Experiments, however, show that our algorithm always achieves the optimal CS -count, while Glaudell et al.'s rarely achieves the optimal K -count.

Remark V.11. Theorem V.9 and Lemma IV.7 together yield $l-2 \leq \text{cs}(A) \leq 2l+1$, comparable to Proposition 6.4 of [17], which gives $\frac{l-3}{2} \leq \text{cs}(A) \leq 2l+2$. Our method tightens the lower bound by a factor of 2.

B. K -optimality of Glaudell et al.'s algorithm

We close this section by showing that the algorithm of Glaudell et al. [17] is K -near-optimal. The original work proved CS -optimality but did not consider the K -count. CS -optimality implies that for any $A \in CCS$,

$$\text{cs}(A) - 1 \leq \text{rk}(G) \leq 2\text{cs}(A) + 2,$$

where G is their output, which has the form of Equation (2) with optimal CS -count, and each Clifford part between two CS gates has K -count 1 or 2. Let E be such a Clifford part. First, $\text{kc}(E) \neq 0$: if E were a generalized permutation (which contains no CS gate since E is Clifford), then the two flanking CS gates in $CS \cdot E \cdot CS$ would cancel, contradicting CS -optimality — cancellation that follows by direct calculation over the finitely many generalized permutations. The bound $\text{kc}(E) \leq 2$ follows by enumerating the two-qubit Cliffords. Hence Glaudell et al.'s algorithm produces circuits with K -count asymptotically at most twice optimal.

VI. EXPERIMENT

We implemented our algorithm in Haskell and compared it with the CS -optimal synthesis algorithm of

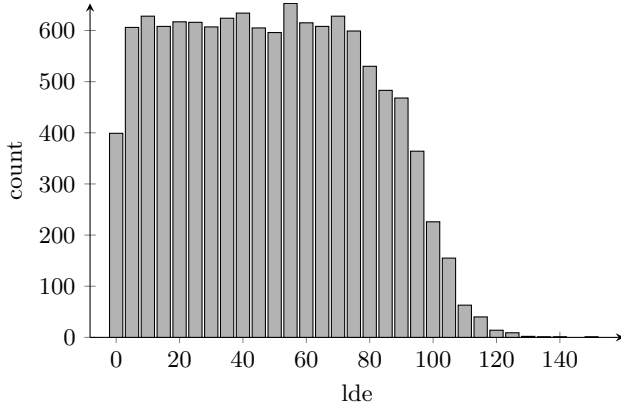


FIG. 2. Distribution of the least denominator exponent (lde) across the 12,000 test operators, binned in groups of 5. The lde ranges from 0 to 152, with a mean of 51 and a median of 50.

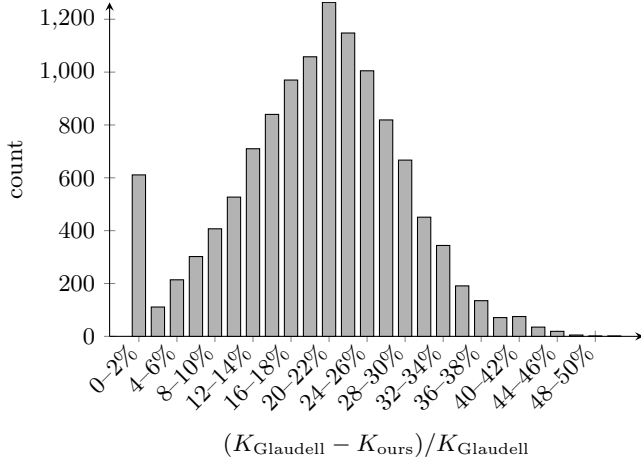


FIG. 3. Distribution of the relative K -count reduction $(K_{\text{Glaudell}} - K_{\text{ours}})/K_{\text{Glaudell}}$, binned in 2% increments. Our method improves on Glaudell et al.'s K -count in 95.1% of cases, with a mean reduction of 19.7%, a median of 20.3%, and a maximum of 50.0%.

Glaudell et al. [17]. We generated 12,000 random two-qubit Clifford+ CS operators with lde ranging from 0 to 152, fed each to both algorithms, and compared the output circuits on `rcs` and `rkc`; the data are available in Section VIC. Since the output of Glaudell et al.'s algorithm has the form of Equation (2) with optimal CS -count, we augmented their algorithm with a K -count-minimizing post-processing step: each C_i is replaced by an equivalent circuit of minimum K -count (0, 1, or 2). Figure 2 shows the distribution of lde values, and Figure 3 shows the distribution of the relative K -count reduction.

The two algorithms produced identical CS -counts on every operator in the test set, so our algorithm is also CS -optimal in practice. Our K -count is strictly smaller than Glaudell et al.'s in 95.1% of cases (Figure 3).

A. Source of improvement

The data show that Glaudell et al.'s K -count is well approximated by twice the CS -count,

$$K_{\text{Glaudell}} \approx 2 \cdot CS\text{-count},$$

and, consistent with Lemma IV.7, that our K -count is well approximated by twice the lde ,

$$K_{\text{ours}} \approx 2 \cdot \text{lde}.$$

The absolute K -count saving is therefore approximately $2 \cdot (CS\text{-count} - \text{lde})$. The relative reduction is roughly stable across lde values, hovering around 19–21% for lde between 5 and 80 and tapering off at the extremes (possibly due to smaller sample sizes).

B. Worst cases

For 588 of the 12,000 operators (4.9%), our method achieves no K -count reduction (Figure 3). The family $W_n = (CK \cdot KC)^n$ is one of the worst cases: a direct computation gives $\text{lde}(W_n) = \text{cs}(W_n)$, and on this family our algorithm and Glaudell et al.'s produce the same K -count, $2 \cdot \text{lde}(W_n)$.

C. Code and data availability

Code and data are available at [7].

VII. CONCLUSION AND FUTURE WORK

We have presented an efficient, K -optimal, CS -near-optimal algorithm for the exact synthesis of two-qubit Clifford+ CS circuits from a given unitary matrix. The theoretical worst-case CS -count is twice optimal, but in practice our method always achieves the optimal CS -count and consistently produces a smaller K -count than the best previously known algorithm.

Several directions remain open. The algorithm of Li et al. [29] produces circuits with T -count at most 10 times optimal in the worst case; we believe our method can be adapted to tighten this bound. Total gate count is another natural target, since experiments show that the raw length of our output is well below the theoretical bound. Extending to the three-qubit case is plausible, though the number of residue patterns is likely far greater than the six in Lemma IV.1, which may complicate the analysis and weaken the bounds; the four-qubit case is harder still. Extending to the two-qubit Clifford+ V and Clifford+Cyclotomic gate sets is a promising avenue. Finally, since our method achieved CS -optimality on all 12,000 randomly generated test operators, we conjecture that it is in fact CS -optimal, and we leave the proof to future work.

VIII. ACKNOWLEDGMENTS

The authors acknowledge support from the National Natural Science Foundation of China under Grant

92465202. Claude Opus 4.6 was used to improve the wording of the paper, generate the distribution diagrams (Figures 2 and 3), and draft initial versions of the Haskell file `Glaude11.hs`. All AI-generated content was verified by the authors.

-
- [1] N. Abdessaied, M. Soeken, and R. Drechsler. Quantum circuit optimization by Hadamard gate reduction. In *Reversible Computation*, 2014.
 - [2] M. Amy, A. N. Glaudell, S. Kelso, W. Maxwell, S. S. Mendelson, and N. J. Ross. Exact synthesis of multiqubit Clifford-Cyclotomic circuits, 2024. arXiv:2311.07741.
 - [3] M. Amy, A. N. Glaudell, S. M. Li, and N. J. Ross. Improved synthesis of Toffoli-Hadamard circuits, 2023. arXiv:2305.11305.
 - [4] M. Amy, A. N. Glaudell, and N. J. Ross. Number-Theoretic Characterizations of Some Restricted Clifford+T Circuits. *Quantum*, 2020.
 - [5] M. Amy, D. Maslov, and M. Mosca. Polynomial-time T-Depth optimization of Clifford+T circuits via matroid partitioning. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2014.
 - [6] X. Bian. EucDomain. <https://github.com/onestruggler/EucDomain>.
 - [7] X. Bian and Y. Feng. Data and code for K-optimal and CS-near-optimal Exact Synthesis of Two-Qubit Clifford+CS Operators. <https://github.com/onestruggler/Kopt>.
 - [8] A. Clement. Private communications.
 - [9] N. de Beaudrap, X. Bian, and Q. Wang. Fast and effective techniques for T -count reduction via spider nest identities. In *15th Conference on the Theory of Quantum Computation, Communication and Cryptography (TQC 2020)*, 2020.
 - [10] N. de Beaudrap, X. Bian, and Q. Wang. Techniques to reduce $\pi/4$ -parity-phase circuits, motivated by the ZX calculus. *Electronic Proceedings in Theoretical Computer Science*, 2020.
 - [11] L. Dinh and N. J. Ross. Contributions to the theory of Clifford-cyclotomic circuits. *Electronic Proceedings in Theoretical Computer Science*, 2025.
 - [12] S. Forest, D. Gosset, V. Kliuchnikov, and D. McKinnon. Exact synthesis of single-qubit unitaries over Clifford-cyclotomic gate sets. *Journal of Mathematical Physics*, 2015.
 - [13] B. Giles and P. Selinger. Exact synthesis of multiqubit Clifford+ T circuits. *Physical Review A*, 2013.
 - [14] B. Giles and P. Selinger. Remarks on Matsumoto and Amano’s normal form for single-qubit Clifford+ T operators. 2013. arXiv:1312.6584.
 - [15] A. N. Glaudell, M. Jarret, S. Klein, S. S. Mendelson, T. C. Mooney, and M. Tian. High-performance exact synthesis of two-qubit quantum circuits, 2026. arXiv:2601.19166.
 - [16] A. N. Glaudell, N. J. Ross, and J. M. Taylor. Canonical forms for single-qutrit Clifford+ T operators. *Annals of Physics*, 2019.
 - [17] A. N. Glaudell, N. J. Ross, and J. M. Taylor. Optimal two-qubit circuits for universal fault-tolerant quantum computation. *npj Quantum Information*, 2021.
 - [18] A. N. Glaudell, N. J. Ross, J. van de Wetering, and L. Yeh. Qutrit Metaplectic Gates Are a Subset of Clifford+ T . In *17th Conference on the Theory of Quantum Computation, Communication and Cryptography (TQC 2022)*, 2022.
 - [19] A. N. Glaudell, N. J. Ross, J. van de Wetering, and L. Yeh. Exact synthesis of multiqutrit Clifford-cyclotomic circuits. *Electronic Proceedings in Theoretical Computer Science*, 2024.
 - [20] L. E. Heyfron and E. T. Campbell. An efficient quantum compiler that reduces T count. *Quantum Science and Technology*, 2018.
 - [21] K. Ireland and M. Rosen. Graduate Texts in Mathematics 84. 1982.
 - [22] A. Jain, A. R. Kalra, and S. Prakash. A normal form for single-qutrit Clifford+ T operators, 2020. arXiv:2011.07970.
 - [23] A. R. Kalra, M. Saikia, D. Valluri, S. Winnick, and J. Yard. Multi-qutrit exact synthesis, 2024. arXiv:2405.08147.
 - [24] A. Kissinger and J. van de Wetering. Reducing the number of non-Clifford gates in quantum circuits. *Phys. Rev. A*, 2020.
 - [25] V. Kliuchnikov. Synthesis of unitaries with Clifford+ T circuits, 2013. arXiv:1306.3200.
 - [26] V. Kliuchnikov, D. Maslov, and M. Mosca. Fast and efficient exact synthesis of single qubit unitaries generated by Clifford and T gates. *Quantum Information and Computation*, 2013.
 - [27] V. Kliuchnikov and S. Schönnenbeck. Multi-qubit circuit synthesis and Hermitian lattices, 2024. arXiv:2405.19302.
 - [28] V. Kliuchnikov and J. Yard. A framework for exact synthesis, 2015. arXiv:1504.04350.
 - [29] L. Li, C. Guo, Q. Li, and X. Sun. Fast exact synthesis of two-qubit unitaries using a near-minimum number of T gates. *Phys. Rev. A*, 2023.
 - [30] K. Matsumoto and K. Amano. Representation of quantum circuits with Clifford and $\pi/8$ gates, 2008. arXiv:0806.3834.
 - [31] P. Mukhopadhyay. Synthesis of V -count-optimal quantum circuits for multiqubit unitaries. *Physical Review A*, 2024.
 - [32] M. A. Nielsen and I. L. Chuang. *Quantum Computation and Quantum Information*. 2002.
 - [33] S. Prakash, A. Jain, B. Kapur, and S. Seth. Normal form for single-qutrit Clifford+ T operators and synthesis of single-qutrit gates. *Phys. Rev. A*, 2018.
 - [34] T. Russell. The exact synthesis of 1- and 2-qubit Clifford+ T circuits, 2014. arXiv:1408.6202.
 - [35] P. Selinger. Generators and relations for n -qubit Clifford operators. *Logical Methods in Computer Science*, 2015.